Implémentation d'un CNN en C et CUDA pour la reconnaissance de chiffres manuscrits

Travail de maturité

Nathan Gasser, 3M4
Répondant TM : Grégoire Favre

19 octobre 2026
Troisième année de maturité

Résumé

Les réseaux de neurones convolutifs (CNN) sont très utilisés de nos jours pour la reconnaissance d'images. Nous allons en implémenter un en utilisant les langages C et CUDA. Il aura pour but de lire un chiffre manuscrit et sera entraîné sur le dataset MNIST, en utilisant une carte graphique NVIDIA. A la fin, nous obtiendrons un modèle capable de lire un chiffre manuscrit avec une précision atteignant 99.44

Table des matières

1 Introduction **4**

2 Avertissement **4**

3 Modèle de reconnaissance d'images **4**

4 MLP **5**
4.1 Perceptron 5
4.2 Fonctions d'activation 6
4.2.1 La fonction $ReLU$ 6
4.2.2 La fonction $tanh$ 6
4.3 Plusieurs neurones 7

5 Backpropagation **7**
5.1 Perte 8
5.2 Backpropagation 8
5.2.1 SGD 9

6 CNN **10**
6.1 Convolution 10
6.2 Pooling 11
6.3 CNN complet 11

7 Dropout **11**

8 Architecture du modèle final **12**

9 Implémentation en C et CUDA **12**
9.1 Moteur d'entrainement en CUDA 12
9.2 Moteur d'inférence en C 13
9.3 Interface utilisateur 13

10 Conclusion **13**

1 Introduction

Pour un humain, être capable de lire un chiffre manuscrit paraît être une tâche évidente. Cependant, pour un ordinateur, la tâche est bien plus complexe que pour un humain, car cette dernière revient à créer un modèle purement mathématique qui reçoit une liste de pixels et qui devrait comprendre comment ces différents pixels s'associent ensemble pour que finalement le modèle sorte un unique chiffre. Pour résoudre ce problème, il y a plusieurs méthodes possibles. Une des méthodes les plus simples est le perceptron à plusieurs couches (MLP), mais les CNN sont plus efficaces pour les tâches de reconnaissance d'images. Un CNN est un type de réseau de neurones qui a pour particularité d'utiliser des opérations de convolution pour extraire des caractéristiques de l'image. Ce TM aura pour but d'implémenter ce type de réseau de neurones en CUDA pour la partie entrainement et par la suite en C pour la partie inférence.

2 Avertissement

Dans ce texte, nous allons utiliser une indexation à partir de 0 et non pas à partir de 1. Dans le contexte des mathématiques, il est plus courant de commencer à partir de 1, mais dans le contexte de l'informatique, 0 est plus souvent utilisé comme point de départ. Ainsi, si nous avons par exemple un vecteur de 3 scalaires ainsi : $v = \begin{bmatrix} 6 \\ 4 \\ 8 \end{bmatrix}$, la valeur v_1 est 4 et la valeur de v_0 est 6.

3 Modèle de reconnaissance d'images

Pour faire un modèle qui est capable de lire un chiffre manuscrit, il faut d'abord comprendre sous quelle forme donner l'image du chiffre manuscrit et également sous quelle forme se présentera la sortie du modèle.

Chaque chiffre manuscrit est une image de 28×28 pixels avec à chaque fois 256 différentes intensités de gris (de complètement blanc à complètement noir). C'est-à-dire que si la valeur du pixel est de 0, il correspond à un pixel complètement blanc et si la valeur du pixel est de 255, il correspond à un pixel complètement noir. Les valeurs entre deux correspondent à différentes intensités de gris. Notez que la valeur maximale d'un pixel correspond à 255 et non 256, tout simplement parce qu'il y a 256 valeurs différentes et que la valeur 0 est incluse, ce qui nous laisse 255 valeurs positives. Nous allons ensuite diviser tous les pixels par 255, ce qui nous permet d'avoir des valeurs entre 0.0 et 1.0 que nous allons pouvoir donner au modèle. Nous avons donc $28 \times 28 = 784$ pixels dans une image. Notre modèle aura donc $28 \times 28 = 784$ entrées avec des valeurs comprises entre 0.0 et 1.0 à chaque fois.

Nous voulons obtenir en sortie 10 probabilités, une pour chacun des chiffres possibles (de 0 à 9). C'est-à-dire que chaque chiffre sera associé à une probabilité que ce soit ce chiffre là qui est écrit sur l'image fournie au modèle. Il y aura donc 10 neurones en sortie, chacun des différents chiffres sera assigné à un neurone et les valeurs

de sortie des neurones se situeront entre -1.0 et 1.0. Une valeur de -1.0 correspond à une certitude qu'il ne s'agit pas du chiffre associé à l'image et 1.0 correspond à une certitude qu'il s'agit du bon chiffre.

4 MLP

L'architecture la plus simple qu'on peut utiliser est le perceptron à plusieurs couches (MLP), il est composé de plusieurs couches composées elles-mêmes de plusieurs perceptrons. Bien qu'elle soit ancienne, elle est toujours utilisée elle-même à l'intérieur de beaucoup d'autres architectures comme le CNN.

4.1 Perceptron

Le perceptron est la brique de base de l'intelligence artificielle. Il accepte un nombre fixe d'entrées (inputs) (N) et nous donne un seul nombre en sortie (output). Le perceptron peut en lui-même déjà être utilisé pour créer des systèmes de classification simples.

Chaque entrée s'appelle x_n et est représentée sous forme d'un nombre rationnel. Pour chaque entrée, il y a un poids (weight) associé (w_n). En plus des entrées et des poids, il y a un biais (bias) (b) associé à chaque neurone. Par convention, ces poids et biais seront appelés paramètres. Ces paramètres sont initialisés de manière aléatoire. Pour calculer la valeur de sortie du perceptron (y), il faut également choisir une fonction d'activation (f). Il peut s'agir théoriquement d'à peu près n'importe quelle fonction mathématique, mais dans le cadre de ce TM, nous allons utiliser deux fonctions principalement : ReLU et tanh.

Pour calculer la valeur de sortie de notre perceptron, il faudra d'abord calculer la valeur z en faisant une somme pondérée des entrées et du biais. Ensuite, nous pourrons appliquer la fonction d'activation à la valeur. La formule est la suivante :

$$y = f(z) = f\left(\sum_{n=0}^{N-1} w_n x_n + b\right)$$

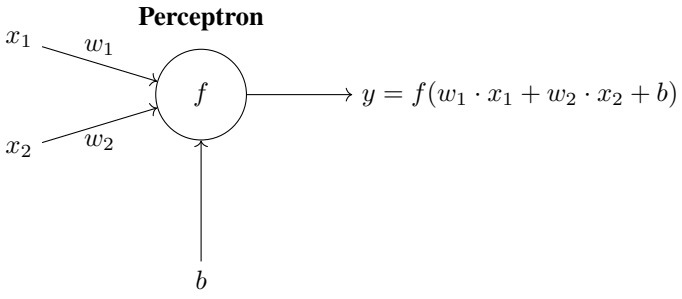


FIGURE 1 – Schéma d'un perceptron avec deux entrées (x_1 et x_2), deux poids (w_1 et w_2), un biais (b) et une fonction d'activation (f) qui produit la sortie (y).

4.2 Fonctions d'activation

La fonction d'activation permet d'ajouter de la non linéarité au modèle. Cela permet au modèle d'apprendre des motifs bien plus complexes. Sans fonction d'activation, le modèle ne pourrait apprendre que des motifs linéaires, ce qui est beaucoup trop limité pour des tâches de reconnaissance d'images. C'est à dire que sans fonction d'activation, tout modèle aussi complexe qu'il soit pourrait être représenté par un seul perceptron. Il est donc très important d'utiliser une fonction d'activation pour que le modèle puisse apprendre des motifs complexes.

4.2.1 La fonction $ReLU$

La fonction $ReLU$ est la suivante :

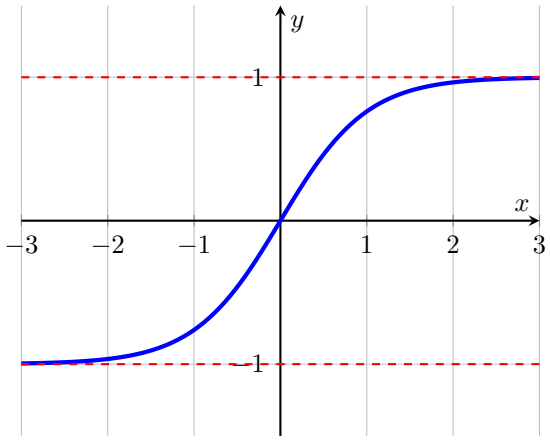
$$ReLU(x) = \begin{cases} 0 \text{ si } x < 0 \\ x \text{ si } x \geq 0 \end{cases}$$

Elle paraît très simple, mais grâce à la non-linéarité qu'elle apporte, elle permet au modèle d'apprendre des motifs très complexes.

4.2.2 La fonction $tanh$

La fonction $tanh$ (tangente hyperbolique) est utilisée pour confiner les valeurs de sorties entre la valeur -1.0 et 1.0. Dans le contexte de la reconnaissance d'images, elle permet de représenter le niveau de certitude de notre modèle par rapport à différentes prédictions. La fonction softmax est également souvent utilisée mais par simplicité j'ai décidé d'utiliser tanh pour ce TM.

Graphe de $tanh(x)$



4.3 Plusieurs neurones

Pour former un MLP complet, il faut agencer plusieurs couches de plusieurs perceptrons ensemble. Sauf pour la couche initiale du modèle, la valeur de l'entrée numéro i de n'importe quel neurone dans la couche numéro c correspondra à la valeur de sortie du neurone i sur la couche $c - 1$. Si nous sommes sur la couche initiale du réseau, les entrées des différents neurones correspondent aux entrées du modèle (dans notre cas, les différents pixels d'une image). Les sorties des différents neurones de la couche finale d'un modèle correspondent aux sorties du modèle (dans notre cas, les différentes probabilités pour chacun des chiffres possibles).

Nous pouvons schématiser un MLP ainsi :

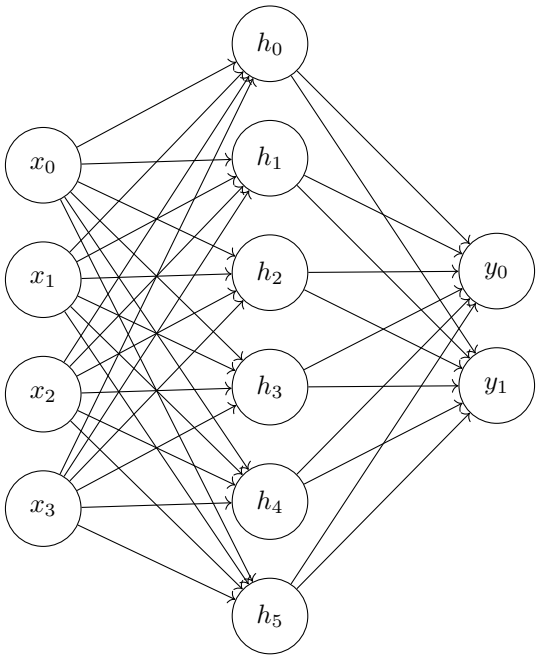


FIGURE 2 – Schéma d'un MLP avec 4 entrées et 2 sorties

5 Backpropagation

Pour que notre modèle puisse correctement prédire les sorties correctes, il va falloir l'entraîner. Sans entrainement, les prédictions seront tout simplement aléatoires. Pour entraîner notre modèle, nous allons utiliser la backpropagation qui est considérée comme l'algorithme le plus important dans le domaine du deep learning. Le but sera de faire la dérivée de la perte par rapport aux différents paramètres du modèle pour savoir comment modifier ces derniers pour la perte diminue et que le modèle fasse donc de meilleures prédictions.

5.1 Perte

Avec la backpropagation, notre but sera de minimiser la perte (loss) (L) du modèle. Si avec une certaine entrée, on obtient pour la sortie numéro n une valeur de y_n, alors que nous voulions obtenir une valeur de l_n, nous pouvons calculer la perte d'une sortie en soustrayant la valeur que nous attendions à la valeur que nous avons obtenue et élever le tout au carré. Cet algorithme s'appelle l'erreur quadratique (Mean Squared Error ou MSE en anglais). Nous pouvons ainsi calculer la perte d'une sortie spécifique L_n de la manière suivante :

$$L_n = (y_n - l_n)^2$$

Pour calculer la perte moyenne d'un modèle avec N sorties, il suffit d'additionner toutes les pertes de chaque sortie et diviser le tout par N :

$$L = \frac{1}{N} \sum_{n=0}^{N-1} (y_n - l_n)^2$$

Calculer la perte elle-même sert uniquement à suivre la qualité du modèle (plus la perte est basse, meilleures sont les prédictions du modèle). Comme expliqué plus haut, ce qui va vraiment nous intéresser, c'est la dérivée de la perte par rapport aux différents paramètres du modèle. C'est ici qu'entre en jeu la backpropagation. Pour calculer la dérivée de la perte par rapport aux différents poids et biais, nous allons utiliser la règle de la chaîne (chain rule). Pour commencer la chaîne, nous allons devoir calculer la dérivée des pertes par rapport aux différentes sorties associées du modèle. Pour une sortie y_n associée à une valeur attendue l_n, la dérivée de la perte totale L par rapport à cette sortie est la suivante :

$$\frac{\partial L}{\partial y_n} = 2 \cdot (y_n - l_n)$$

5.2 Backpropagation

Ensuite, nous allons pouvoir utiliser la règle de la chaîne pour faire la backpropagation et calculer la dérivée de la perte par rapport aux différents paramètres du modèle. Pour commencer, il va falloir calculer la dérivée de la perte par rapport aux valeurs z. Pour un neurone n dont la valeur de sortie équivaut à y_n, la dérivée de la perte par rapport à la valeur z est la suivante si la fonction d'activation est $tanh$:

$$\frac{\partial L}{\partial z_n} = \frac{\partial L}{\partial y_n} \cdot \frac{\partial y_n}{\partial z_n} = \frac{\partial L}{\partial y_n} \cdot (1 - y_n^2)$$

et si la fonction d'activation est $ReLU$:

$$\frac{\partial L}{\partial z_n} = \frac{\partial L}{\partial y_n} \cdot \frac{\partial y_n}{\partial z_n} = \begin{cases} 0 \text{ si } z_n < 0 \\ \frac{\partial L}{\partial y_n} \text{ si } z_n \geq 0 \end{cases}$$

Ensuite, nous allons faire la dérivée à travers la valeur z pour avoir la dérivée de la perte par rapport aux poids et aux biais. Pour un poids w_{wn} associé à une entrée x_{in}, la dérivée de la perte par rapport à ce poids est la suivante :

$$\frac{\partial L}{\partial w_{wn}} = \frac{\partial L}{\partial z_n} \cdot \frac{\partial z_n}{\partial w_{wn}} = \frac{\partial L}{\partial z_n} \cdot x_{in}$$

Et pour un biais b_n associé à un neurone n, la dérivée de la perte par rapport à ce biais est la suivante :

$$\frac{\partial L}{\partial b_n} = \frac{\partial L}{\partial z_n} \cdot \frac{\partial z_n}{\partial b_n} = \frac{\partial L}{\partial z_n}$$

Pour un neurone dans la couche finale du modèle, nous avons déjà montré comment cette dérivée se calcule. Pour un neurone dans une autre couche, il va falloir faire la somme pondérée des dérivées des neurones de la couche suivante. Pour un neurone n de la couche c et M neurones dans la couche $c+1$, la dérivée de la perte de cette sortie spécifique par rapport à la valeur de sortie de ce neurone est la suivante :

$$\frac{\partial L}{\partial y_{c,n}} = \sum_{m=0}^{M-1} \frac{\partial L}{\partial z_{c+1,m}} \cdot w_{c+1,m,n}$$

Les dérivées des neurones des couches précédant la couche de sortie nécessitent de connaître les dérivées des neurones de la couche qui suit ces dits neurones, il est donc nécessaire de faire la backpropagation en calculant d'abord les dérivées des neurones de la couche de sortie, puis celles de la couche précédente et ainsi de suite jusqu'à la couche initiale.

5.2.1 SGD

Maintenant que nous avons les dérivées de la perte par rapport aux différents paramètres du modèle, nous allons devoir mettre à jour ces paramètres tel que la perte diminue. Pour ce faire, nous allons utiliser l'algorithme de descente de gradient stochastique (Stochastic Gradient Descent ou SGD en anglais). Cet algorithme est très simple à comprendre. Si nous connaissons la dérivée de la perte par rapport à un paramètre pi_n, nous savons que si nous changeons la valeur de ce paramètre dans le sens opposé à la dérivée, la perte va diminuer. Étant donné que nous ne voulons pas changer la valeur de ce paramètre trop rapidement, car nous pourrions ainsi dépasser le minimum de la fonction de perte, nous allons multiplier la dérivée par un scalaire appelé le taux d'apprentissage (learning rate) (α). Dans le cadre de ce TM, sa valeur est de 0.05. La formule pour déterminer le nouveau paramètre (pf_n) est la suivante :

$$pf_n = pi_n - \alpha \cdot \frac{\partial L}{\partial pi_n}$$

6 CNN

En utilisant l'architecture MLP, nous avons pu créer un modèle capable de reconnaitre des chiffres manuscrits. Cependant, il est possible de créer un modèle plus efficace en utilisant l'architecture CNN. Le CNN va utiliser des filtres pour extraire des caractéristiques de l'image. Ces filtres vont être réutilisés pour plusieurs pixels de l'image, ce qui permet de réduire drastiquement le nombre de paramètres, permettant de réduire les ressources nécessaires à l'entrainement et l'inférence.

6.1 Convolution

La particularité principale d'un CNN est l'utilisation des couches de convolution. Une opération de convolution va appliquer un filtre (kernel ou filter en anglais) sur notre entrée. Le filtre et l'entrée sont tous deux représentés sous forme de matrices. Cela permet au modèle de comprendre des motifs en 2d dans l'image. Le filtre est une matrice bien plus petite que l'image d'entrée, de dimensions $sf \times sf$. Notez que les valeurs du filtre sont générées aléatoirement et font partie des paramètres du modèle à optimiser avec la backpropagation. La matrice d'entrée est de dimension $si \times si$. Dans le cadre du TM, la sortie sera représentée sous forme d'une matrice de dimensions $(si - sf + 1) \times (si - sf + 1)$. Pour calculer la valeur d'une cellule de sortie $y_{i,j}$ avec une matrice d'entrée x et un filtre f, voici la formule à utiliser :

$$y_{i,j} = \sum_{ii=0}^{sf-1} \sum_{ji=0}^{sf-1} x_{i+ii,j+ji} \cdot f_{ii,ji}$$

Dans une couche de convolution, il y a plusieurs filtres qui peuvent être utilisés. Chaque filtre va produire une sortie différente, chaque sortie s'appelant un canal (channel). Si nous avons plusieurs canaux de sortie, vous noterez qu'il faudra également accepter plusieurs canaux d'entrée. Si nous avons nc canaux d'entrée avec nf filtres (donc également nf canaux de sortie), nous devons utiliser un nombre nf de filtres et une entrée représentés par des tenseurs d'ordre 3. Ainsi, nous avons nf filtres de dimensions $nc \times sf \times sf$ et une entrée de dimensions $nc \times si \times si$. La sortie sera donc représentée par un tenseur de dimensions $nf \times (si - sf + 1) \times (si - sf + 1)$. Nous pouvons également représenter tous les filtres sous forme d'un tenseur d'ordre 4 de dimensions $nf \times nc \times sf \times sf$. Pour calculer la valeur d'une cellule de sortie $y_{ci,i,j}$ avec notre tenseur d'ordre 4 de filtres f et notre tenseur d'ordre 3 pour nos entrées x, voici la formule à utiliser :

$$y_{ci,i,j} = \sum_{ici=0}^{nc-1} \sum_{ii=0}^{sf-1} \sum_{ji=0}^{sf-1} x_{ici,i+ii,j+ji} \cdot f_{ci,ici,ii,ji}$$

Nous pouvons écrire la dérivée de la perte par rapport au poids d'un filtre $f_{ci,ici,fi,fj}$ de la manière suivante, avec so la taille de la sortie :

$$\frac{\partial L}{\partial f_{ci,ici,fi,fj}} = \sum_{i=0}^{so-1} \sum_{j=0}^{so-1} \frac{\partial L}{\partial y_{ci,i,j}} \cdot x_{ici,i+fi,j+fj}$$

Et la dérivée de la perte par rapport à une entrée $x_{ici,i,j}$ de la manière suivante :

$$\frac{\partial L}{\partial x_{ici,i,j}} = \sum_{ci=0}^{nc-1} \sum_{fi=0}^{sf-1} \sum_{fj=0}^{sf-1} \frac{\partial L}{\partial y_{ci,i-fi,j-fj}} \cdot f_{ci,ici,fi,fj}$$

6.2 Pooling

Il existe également des couches de pooling qui permettent de réduire la taille des entrées sans perdre trop d'informations. Il existe plusieurs manières de faire du pooling, mais dans le cadre de ce TM, nous allons utiliser le max pooling. Le max pooling nécessite de déterminer à l'avance une pool size (ps) et prendra comme entrée une matrice de dimensions $si \times si$ pour produire une matrice de dimensions $(si/ps) \times (si/ps)$. Les couches de pooling n'ont pas de paramètres à entrainer. La valeur d'une cellule de sortie $y_{i,j}$ est la valeur maximale dans la sous-matrice de dimensions $ps \times ps$ de l'entrée x commençant à la position $i \cdot ps, j \cdot ps$ et finissant à la position $(i+1) \cdot ps - 1, (j+1) \cdot ps - 1$.

Pour la dérivée de la perte par rapport à une cellule d'entrée $x_{i,j}$, c'est très simple. Si la cellule d'entrée $x_{i,j}$ est la même que la valeur associée dans la cellule de sortie $y_{i/ps,j/ps}$, alors la dérivée de la perte par rapport à cette cellule d'entrée est la même que la dérivée de la perte par rapport à cette cellule de sortie. Sinon, la dérivée de la perte par rapport à cette cellule d'entrée est égale à 0.

Dans le cadre d'un CNN, il y a souvent plusieurs canaux. Pour gérer plusieurs canaux, c'est très simple, il suffit de faire l'opération de pooling sur chaque canal indépendamment. Ainsi, si nous avons nc canaux d'entrée, nous aurons également nc canaux de sortie.

6.3 CNN complet

Pour créer un CNN complet, nous avons souvent une architecture qui commence par plusieurs suites de couches de convolution et de pooling, suivies par plusieurs couches de MLP.

7 Dropout

Une autre couche qui est très utile dans les CNN est la couche de dropout. Cette couche permet au modèle de ne pas trop dépendre de certains neurones spécifiques, ce qui permet au final d'avoir un modèle de meilleure qualité. Le dropout consiste à désactiver aléatoirement un certain pourcentage de neurones dans une couche (dropout rate) (dr). Pour une entrée x avec N scalaires, nous allons désactiver aléatoirement $N \cdot dr$ scalaires en mettant leur valeur de sortie à 0. Pour les scalaires qui ne sont pas désactivés, nous allons diviser leur valeur de sortie par $(1 - dr)$ pour compenser le fait que certains neurones ont été désactivés. Ainsi, la somme des valeurs de sortie sera la même que si aucun neurone n'avait été désactivé.

Pour calculer la dérivée de la perte par rapport à une cellule d'entrée x_n, si cette cellule d'entrée a été désactivée, la dérivée de la perte par rapport à cette cellule d'entrée est égale à 0. Sinon, elle est égale à la dérivée de la perte par rapport à cette cellule de sortie divisée par $(1 - dr)$.

Notez que le dropout n'est utilisé que pendant l'entrainement du modèle. Pendant l'inférence, aucun neurone n'est désactivé et les valeurs de sortie ne sont pas divisées par $(1 - dr)$.

8 Architecture du modèle final

Notez que dans l'architecture du modèle final, il y a également des couches de ReLU après les couches de pooling. Cela veut dire que les valeurs de sortie des couches de pooling sont passées à travers la fonction ReLU avant d'être données à la couche suivante. Pour la dérivée de la perte par rapport à la valeur d'entrée de la couche ReLU, le concept reste le même que pour la fonction d'activation ReLU dans un perceptron. Si la valeur d'entrée est négative, la dérivée de la perte par rapport à cette valeur d'entrée est égale à 0. Sinon, elle est égale à la dérivée de la perte par rapport à la valeur de sortie.

Voici l'architecture finale de notre modèle couche par couche :

1. Couche de convolution avec 28x28 pixels d'entrée et 32 filtres de taille 3x3.
2. Couche de pooling avec 32 canaux d'entrée, une pool size de 2 suivie d'une couche ReLU.
3. Couche de dropout avec un dropout rate de 0.25.
4. Couche de convolution avec 32 canaux d'entrée de taille 13x13 chacun et 64 filtres de taille 4x4.
5. Couche de pooling avec 64 canaux d'entrée, une pool size de 2 suivie d'une couche ReLU.
6. Couche de dropout avec un dropout rate de 0.25.
7. Couche de MLP avec $64 \cdot 5 \cdot 5 = 1600$ entrées et 128 neurones suivie d'une couche ReLU.
8. Couche de dropout avec un dropout rate de 0.5.
9. Couche de MLP avec 128 entrées et 10 neurones suivie d'une couche tanh.

9 Implémentation en C et CUDA

9.1 Moteur d'entrainement en CUDA

Pour entrainer le modèle, nous allons utiliser le langage CUDA. C'est un langage de programmation développé par NVIDIA qui permet d'utiliser la puissance des cartes graphiques NVIDIA pour effectuer beaucoup de calculs en parallèle. Le parallélisme est très utile dans l'entrainement d'un modèle de deep learning, car beaucoup de calculs

sont indépendants les uns des autres tout en suivant un même algorithme, ce qui nous permet de les effectuer tout en gagnant beaucoup de temps.

Pour l'entrainement du modèle, nous allons utiliser le dataset MNIST. Il contient 60'000 images de chiffres manuscrits pour l'entrainement et 10'000 images pour tester les performances du modèle. Chaque image est donc représentée par une matrice de dimensions 28×28 avec des valeurs entre 0 et 255. Chaque image est également associée à un label qui correspond au chiffre représenté sur l'image. Le label est représenté par un entier entre 0 et 9.

Avec le code final que vous pouvez trouver sur mon repository GitHub, j'ai pu entrainer le modèle sur le dataset MNIST pendant presque 2 heures sur mon ordinateur avec une carte graphique NVIDIA GeForce GTX 1660 super. Le code a 100 epochs, c'est-à-dire qu'il va faire entrainer le modèle 100 fois sur chacune des 60'000 images en tout. Au pic de performance du modèle, il a pu atteindre une précision de 99.44% sur le dataset de test. Cela veut dire que sur les 10'000 images de test, le modèle a pu correctement reconnaitre 9'944 images et s'est trompé sur uniquement 56 images.

9.2 Moteur d'inférence en C

Le désavantage du CUDA est qu'il ne fonctionne que sur un ordinateur muni d'une carte graphique NVIDIA. C'est pour cela que j'ai également écrit le code pour l'inférence du modèle en C qui peut lui être exécuté sur à peu près n'importe quel ordinateur. Le code est majoritairement composé de copiés-collés du code CUDA avec quelques différences. La vitesse du modèle n'est pas un problème ici, car l'inférence d'un modèle requiert beaucoup moins de calculs que l'entrainement étant donné que l'entrainement requiert également de faire la backpropagation et ce sur $100 \cdot 60'000 = 6'000'000$ images. Le code C pour l'inférence du modèle est également disponible sur mon repository GitHub.

9.3 Interface utilisateur

J'ai également écrit une interface utilisateur en HTML et JavaScript qui permet de dessiner un chiffre manuscrit sur un canvas et de l'envoyer au modèle pour qu'il le reconnaisse. Elle est également disponible sur mon repository GitHub.

10 Conclusion

En conclusion, j'ai pu implémenter avec succès ce réseau de neurones convolutif. Il m'a permis d'approfondir mes connaissances dans le domaine du machine learning après avoir fait mes premiers pas un an auparavant en implémentant un simple MLP sur la même tâche de reconnaissance d'image. Je pense d'ailleurs que j'aurais implémenté cette architecture dans mon temps libre même s'il ne s'agissait pas de mon TM, car j'ai même plus récemment implémenté l'architecture de Transformer, qui est l'architecture phare derrière les modèles de langage tels que ChatGPT ou Claude.